



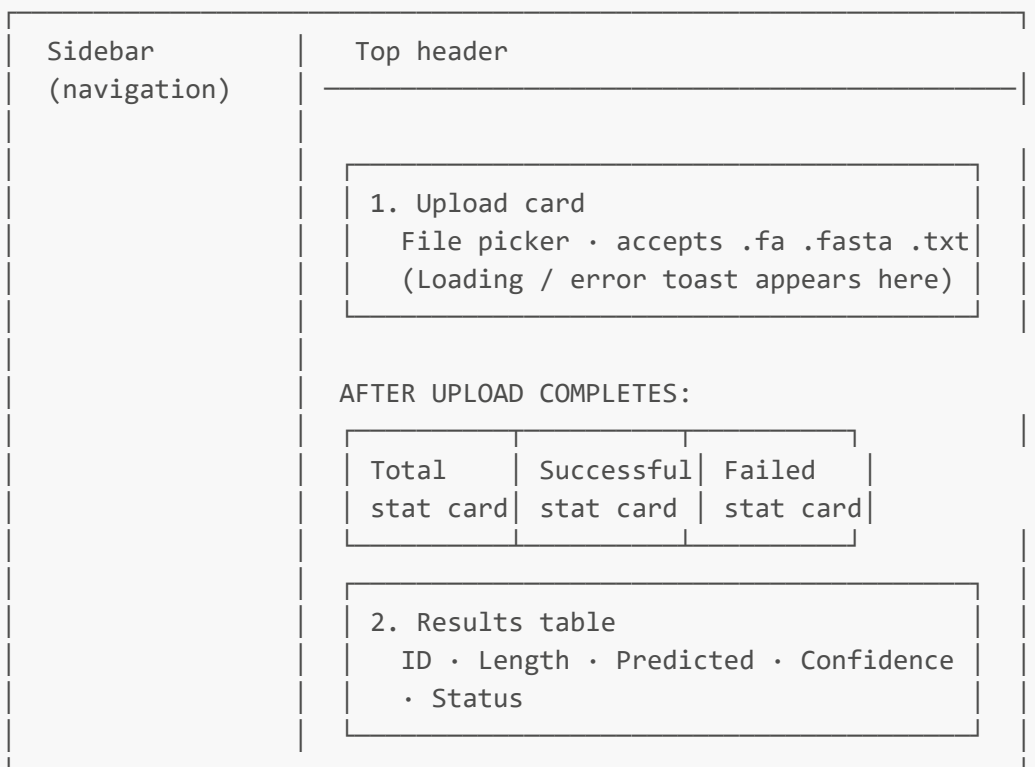
Batch Upload Page — Complete Component Guide

A deep-dive into every panel and control on the **Batch Upload** page of GenomelQ. For each component this document covers:

- **What it is** (plain-language description)
- **What it tells you** (how to read the output)
- **How it works under the hood** (the algorithm)
- **How to use it** (the user flow)
- **Practical tips** (when it's useful, when to be cautious)



Page Layout Overview



The Batch Upload page has a **simple, linear flow**: a single upload card at the top, three stat cards once processing completes, and a table of per-record results below.

1. Upload Card (File Picker)

What it is

The single input control on the page — a styled file picker that accepts FASTA-formatted DNA files.

What it tells you

Element	Meaning
Header text	"Batch FASTA upload" — plus a one-line description of what the page does
File picker	A native HTML <code><input type="file"></code> styled with Tailwind. Accepts <code>.fa</code> , <code>.fasta</code> , <code>.txt</code> extensions.
Loading text (below picker)	"Processing..." shown while the upload is being processed.
Error toast (below picker)	Rose-colored text showing any error returned from the server.

⚙️ How it works under the hood

When you select a file, Alpine fires `onFile($event)`:

```

async onFile(event) {
  const file = event.target.files?.[0];
  const fd   = new FormData();
  fd.append("file", file);

  const r    = await fetch("/batch", { method: "POST", body: fd });
  const data = await r.json();
  this.batch.result = data;
}

```

The server side (`src/api/app.py` → `batch()`):

1. Reads the file as UTF-8 text.
2. Calls `parse_fasta(content)` from `src/core/sequence.py` to extract every record.
3. For each record:
 - Validates the sequence (length, characters).
 - Runs the classifier if valid.
 - Records `{ record_id, description, length, prediction, error }`.
4. Returns aggregated stats `{ total, successful, failed, results }`.

The FASTA parser is forgiving: it accepts **multi-record** files (each starting with `>`), but if the input has no `>` lines at all, the entire content is treated as a single record.

📖 How to use it

1. Prepare a FASTA file:

```

>seq_brca1
ATGGATTTATCTGCTCTTCGCGTT...
>seq_ins
ATGGCCCTGTGGATGCGCCTC...

```

```
>seq_app
ATGCTGCCCCGGTTTGGCACTGCT...
```

2. Click  **Batch Upload** in the sidebar.
3. Click the file picker button.
4. Select your **.fasta** file. Upload starts immediately.
5. Wait for the **"Processing..."** text to disappear (a few hundred ms per record).

Practical tips

- **No size limit imposed** by the front end, but the backend enforces a **200,000-base maximum per sequence**. Files with longer sequences will have those records marked as failed.
- **Compressed files (.gz, .zip) are explicitly rejected** with a helpful error message — uncompress them first.
- The page only supports **one file at a time**. To process multiple files, run them one after another.
- **Empty FASTA files** are caught and rejected with *"Uploaded file is empty."*
- The picker also accepts **plain .txt files** as long as they contain FASTA-formatted content.
- Sequences with a few invalid characters are still tolerated — invalid characters are silently stripped before validation.

2. Summary Stat Cards (Three Cards)

What it is

Three small stat cards displaying aggregate counts over the uploaded batch, shown only **after** processing completes.

What it tells you

Card	Color	Meaning
Total	Default	Number of records detected in the FASTA file.
Successful	Green	Records the classifier predicted without error.
Failed	Red	Records that failed validation or prediction (e.g., too short, invalid characters).

A run with **total = 5**, **successful = 4**, **failed = 1** means 5 records were parsed, 4 got predictions, and 1 had an error (which you'll see in the table below).

How it works under the hood

These come straight from the **/batch** endpoint's JSON:

```
{
  "total": 5,
```

```
"successful": 4,  
"failed": 1,  
"results": [...]  
}
```

The Alpine template just shows them in three pills.

How to use it

A glance gives you a high-level **batch health check**:

- **Successful = Total** → batch ran cleanly.
- **Failed > 0** → scroll down to the table and look for red status messages.

Practical tips

- Even if **Failed > 0**, the **successful records are saved** in the **results** array. The error doesn't break the entire batch.
- For very large batches, the page may briefly freeze while rendering the table — give it a few seconds.

3. Results Table

What it is

A single table listing **every record** in the FASTA file, with one row per sequence.

What it tells you

Column	Meaning
ID	The record identifier from the <code>>id ...</code> line in the FASTA (monospace text).
Length	The length of the cleaned (valid) DNA sequence in bases.
Predicted	Color-coded disease badge (Cancer / Diabetes / Alzheimers / Normal). Empty if the record failed.
Confidence	Probability of the predicted class as a percentage. Dash (–) if failed.
Status	OK (green) for successful records, or a short error message (red) for failed ones.

The table is unstyled — no sorting or pagination — but the visual scan is fast even for hundreds of rows because each row uses a compact pill for the prediction.

How it works under the hood

The Alpine template iterates over `batch.result.results`:

```
<template x-for="row in batch.result.results">  
  <tr>
```

```

<td x-text="row.record_id"></td>
<td x-text="row.length.toLocaleString()"></td>
<td><span x-text="row.prediction.predicted_class"></span></td>
<td x-text="(row.prediction.confidence*100).toFixed(1)+'%"></td>
<td :class="row.error ? 'text-rose-500' : 'text-emerald-500'"
    x-text="row.error || 'OK'"></td>
</tr>
</template>

```

The disease color comes from `diseaseColor()` which reads the same `DISEASE_THEME` map used elsewhere in the UI.

How to use it

1. After the upload, scroll the table.
2. **Quickly scan the Status column** for any red error messages.
3. **Look at the Predicted column** to verify the right classes are appearing.
4. **Sort manually:** copy the table contents into a spreadsheet if you want to sort/filter.

Practical tips

- The table can be **copied wholesale** into Excel/Google Sheets for further analysis. Selecting the entire table and pasting preserves the column structure.
- Common error messages and their meaning:

Error message	Cause
<i>Sequence too short. Minimum 20 bases required.</i>	The FASTA record has fewer than 20 valid bases after cleaning.
<i>No valid DNA bases found.</i>	The record contained only invalid characters (e.g., empty or HTML-pasted).
<i>Sequence too long. Maximum 200,000 bases supported.</i>	Record exceeds the upper limit.
<i>Prediction error: ...</i>	An internal error during classification — open the server log for details.

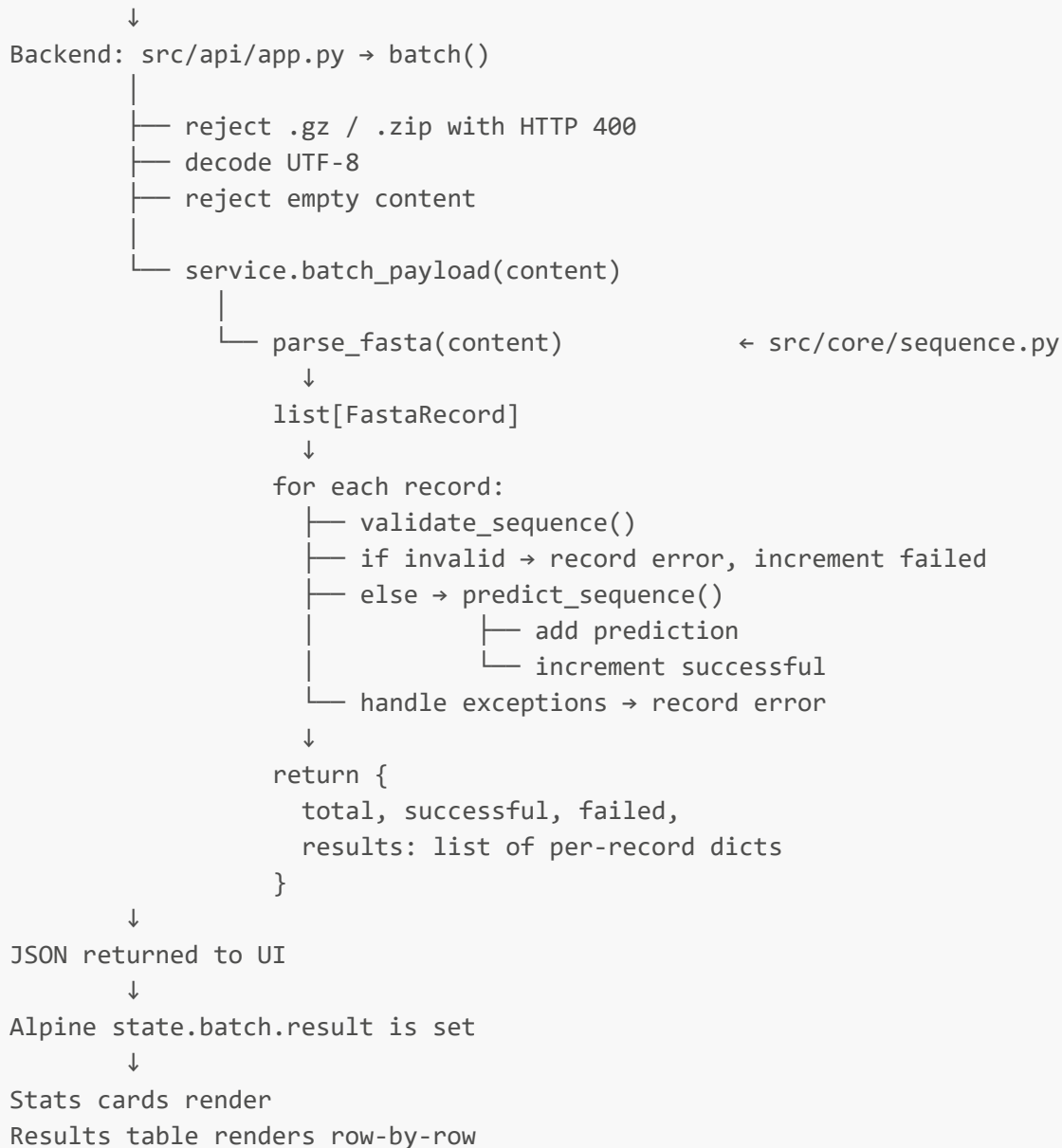
- The display caps at the full results list (up to thousands of rows). If your batch is huge, the response payload may get large; consider splitting into multiple files.

The Full Batch Upload Flow (End-to-End)

```

User selects FASTA file
    ↓
File read into FormData
    ↓
[POST /batch] multipart/form-data: file=...

```



Total time depends on batch size: **~50 ms per record** for TFIDF, more for DNABERT-2 (the batch endpoint always uses the default backbone — currently TFIDF).

Cheat Sheet — Common Workflows

Scenario	What to do
Predict 10 sequences from a single experiment	Save them as a multi-record FASTA, upload once, scan the table.
Find which sequences fall into each class	Upload, copy the table to a spreadsheet, group by Predicted.
Spot OOD sequences	The batch endpoint doesn't run OOD per-record — re-run any suspect one through the Analyze page for the OOD verdict.
Compare model backbones for a whole batch	The batch endpoint hardcodes TFIDF. For per-backbone comparison, run via the API or a custom script (see src/predict.py).

Building a Test FASTA Quickly

If you don't have a FASTA handy, copy this into a new file `test.fasta`:

```
>seq_brca1
ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCCATCTGTC
TGGAGTTGATCAAGGAACCTGTCTCCACAAAGTGTGACCACATATTTTGCAAATTTTGCATGCTGAAACTTCTCAACCAGAA
GAAAGGGCCTTCACAGTGTCTTTATGTAAGAATGATATAACCAAAAGGG
>seq_ins
ATGGCCCTGTGGATGCGCCTCCTGCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACCTGACCCAGCCGAGCCTTTGTGAACC
AACACCTGTGCGGCTCACACCTGGTGGAAGCTCTCTACCTAGTGTGCGGGGAACGAGGCTTCTTCTACACACCCAAGACCCG
CCGGGAGGCAGAGGACCTGCAGGTGGGGCAGGTGGAGCTGGGCGGTGGCCCTGGTGCAGGCAGCCTGCAGCCCTTGGCCCTG
GAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGT
>seq_app
ATGCTGCCCCGTTTGGCACTGCTCCTGCTGGCCGCCTGGACGGCTCGGGCAGTGCAGAATTCTGACATGCTGCAGAATTTCA
GCCAGAATCAACCCGACTTCTCTGACTATGACAACAGCCACAGCAGCCAGCCAGAGCCAGCCATGGAAGATGAGGATGAAGA
CGAAGATGAGGAAGATGAAGACGAGCCCAAAGAGGAAGATGAACCCAGGAAGGAGGAT
>seq_actb
ATGGATGATGATATCGCCGCGCTCGTCGTCGACAACGGCTCCGGCATGTGCAAAGCCGGCTTCGCGGGCGACGATGCCCCGA
GGGCACTCTTCCAGCCTTCCTTCTGGGCATGGAGTCCTGTGGCATCCACGAACTACCTTCACTCCATCATGAAGTGTGA
CGTGACATCCGCAAAGACCTGTACGCCAACACACATGAAGATCAAGATCATTGCTCCTCCTGAGCGCAAGTACTCCGTGTG
GATCGGCGGCTCCATCCTGGCCTCGCTGTCCACCTTCCAG
```

Upload it — you should see all 4 records predicted correctly (Cancer, Diabetes, Alzheimers, Normal).

Related Documents

- [README.md](#) — high-level project overview
- [DOCS.md](#) — comprehensive technical documentation
- [TESTING_GUIDE.md](#) — full test plan including batch upload cases
- [Guide/ANALYZE_PAGE_GUIDE.pdf](#) — predictor walkthrough (predictions reused here)

Part of the GenomelQ per-page documentation series.